METRO TRANSIT SYSTEM

SQL Analytics & Operational Performance Report

Prepared by

R Hemant Kumar Patra

Technology SQL / MySQL

Report Type Data Analytics & Database Project

PROJECT PURPOSE

Translate a relational metro-transit database into operational insights covering passenger demand, route utilization, fare behavior, train performance, station activity, and SQL engineering practices.

Portfolio-ready presentation • Evidence-led analysis • Clean relational design

01 Executive Overview

A concise view of the network, analytical scope, and database foundation.

The Metro Transit System project models a city rail network through four connected relational tables: **Stations**, **Trains**, **Passengers**, and **TicketBookings**. The project is designed to demonstrate SQL from database constraints through advanced analytical techniques.

25	20	465	1,490
STATIONS	TRAINS	PASSENGERS	BOOKINGS

Analytical Scope

Area	Business Question	SQL Focus
Demand	Where is passenger activity concentrated?	GROUP BY, HAVING, JOIN
Revenue	Which routes and seat classes drive value?	SUM, AVG, CASE
Operations	Which trains and stations show stronger activity?	JOIN, aggregation
Passengers	Who are the most active and valuable users?	COUNT, subqueries
Time	How does booking activity change over time?	Date functions

DATA INTEGRITY

The project guide confirms the four-table design and approximate record counts. Revenue, percentages, rankings, and trend findings should be calculated only after loading the complete SQL dataset.

02 Database Architecture

The relational structure supports both operational reporting and advanced SQL analysis.

Core entities

Table	Role in the system	Key fields
Stations	Metro station master data	station_id, station_name, city, zone, opened_year
Trains	Train and route information	train_id, train_name, route_line, capacity
Passengers	Registered passenger information	passenger_id, passenger_name, age, email, city
TicketBookings	Transactional booking records	booking_id, passenger_id, train_id, boarding_station_id, destination_station_id, fa

Relationship map

PARENT / MASTER	RELATIONSHIP	TRANSACTION
Passengers	passenger_id → passenger_id	TicketBookings
Trains	train_id → train_id	TicketBookings
Stations	station_id → boarding_station_id	TicketBookings
Stations	station_id → destination_station_id	TicketBookings

DESIGN HIGHLIGHT

TicketBookings references the Stations table twice: once for the boarding station and once for the destination station. This creates a useful real-world example of multiple foreign-key relationships to the same parent table.

03 Demand & Network Utilization

Frame the analysis around where passengers actually use the network.

Primary question

Where is passenger demand concentrated across routes, trains, and stations?

Analysis	Measure	Recommended presentation
Route-line demand	Bookings + booking share	Ranked bar/table
Train demand	Bookings + revenue	Top / low performers
Boarding activity	Boarding count + revenue	Station ranking
Destination activity	Destination count	Station ranking
Monthly demand	Bookings by month	Trend chart

Analytical interpretation

The strongest demand story should combine booking volume with revenue rather than relying on a single metric. For example, a route can lead in bookings while another generates stronger revenue through a higher average fare.

> **FINDING PLACEHOLDER**
>
> **Populate this callout after running the complete SQL dataset: identify the leading route line, most-used train, and highest-activity boarding station.**

04 Revenue & Fare Performance

Understand how booking volume translates into fare revenue.

Dimension	Core metrics	Decision value
Route line	Bookings, revenue, average fare	Compare demand vs. monetary contribution
Seat class	Bookings, revenue, revenue share	Understand class-level value
Fare category	Low / Medium / High	Profile fare distribution

Suggested fare classification

Category	Rule	Output
Low Fare	Project-defined lower fare band	Booking count + share
Medium Fare	Project-defined middle fare band	Booking count + share
High Fare	Project-defined upper fare band	Booking count + share

BUSINESS LENS

Revenue analysis should distinguish between high-volume and high-value segments. Use actual SQL results to identify whether revenue is concentrated by route, seat class, or fare band.

05 Passenger Behavior & Time Analysis

Move from network performance to customer usage patterns.

Passenger behavior

Metric	SQL analysis
Age profile	MIN, MAX, AVG age
Email completeness	Count missing email values
Repeat usage	Passengers with more than one booking
High-value passengers	Top passengers by total fare paid
Passenger cities	Passenger count by city

Time-based performance

Metric	Analysis
Monthly bookings	COUNT bookings by month
Monthly revenue	SUM fare by month
Average fare trend	AVG fare by month
Travel-time profile	Average / range of travel_time_minutes

> **ANALYTICAL GUARDRAIL**
>
> **Do not claim seasonality from a short observation period. A trend should be described only when the available booking dates provide enough evidence.**

06 SQL Engineering & Analytical Techniques

The project demonstrates a progression from database design to advanced analytical SQL.

Capability	Techniques demonstrated
Database design	PRIMARY KEY, FOREIGN KEY, UNIQUE, DEFAULT, CHECK
Aggregation	COUNT, COUNT(DISTINCT), SUM, AVG, GROUP BY, HAVING
Text & dates	UPPER, LENGTH, MONTH, YEAR, DAYNAME, DATEDIFF, DATE_ADD
Joins	INNER JOIN, LEFT JOIN, multi-table joins
Advanced SQL	Subqueries, CTEs, Views, CASE
Window functions	RANK, DENSE_RANK, LAG, LEAD
Database operations	Transactions, ROLLBACK, indexes, composite indexes

Selected problem-solving pattern

Business Question	SQL Approach
Which trains exceed a booking threshold?	JOIN + GROUP BY + HAVING
Which passengers exceed average spending?	Aggregation + subquery
How do passenger rankings change?	Window functions
How can repeated logic be simplified?	CTE / View
How can frequent searches be optimized?	Indexes / composite indexes

PORTFOLIO VALUE

The project shows more than query writing: it demonstrates relational thinking, aggregation, comparative analysis, reusable SQL structures, window functions, transaction handling, and basic query-performance awareness.

07 Findings, Recommendations & Conclusion

Translate SQL output into operational decision support.

Evidence-led findings

Theme	Final finding to populate from the complete dataset
Demand	Leading route line, train, and station by booking activity
Revenue	Highest-value route and seat/fare category
Operations	Strongest and weakest train/station activity
Passengers	Repeat usage and high-value passenger behavior
Time	Highest / lowest booking month and revenue movement

Recommended actions

Action area	Recommendation principle
Capacity planning	Use sustained booking concentration to guide service review.
Route management	Monitor high- and low-performing lines using both bookings and revenue.
Station operations	Investigate high-demand locations and stations with weak activity.
Revenue strategy	Evaluate fare-class and route-level performance before pricing decisions.
Passenger retention	Use repeat-booking patterns to understand recurring users.
Data quality	Monitor missing passenger contact information and other completeness issues.

Conclusion

The Metro Transit System SQL project provides a practical example of how relational data can be transformed into operational intelligence. By combining passengers, trains, stations, and ticket transactions, the analysis can reveal where demand is concentrated, how revenue is generated, which parts of the network require attention, and how passengers use the service over time. The final report should replace all data-dependent placeholders with verified SQL outputs before publication.

Prepared by	Project	Technology
R Hemant Kumar Patra	**Metro Transit System**	**SQL / MySQL**